

Transformer-Based Models

Transformer-based models are a major breakthrough in Natural Language Processing (NLP).

They were introduced in the paper

“Attention Is All You Need” (Vaswani et al., 2017).

They replaced older sequence models like:

- RNN (Recurrent Neural Networks)
 - LSTM (Long Short-Term Memory)
 - GRU (Gated Recurrent Units)
-

Why Transformers Replaced RNNs / LSTMs

Problems with RNN / LSTM

- Sequential processing → slow
- Hard to parallelize
- Long-range dependencies are difficult
- Memory bottleneck for long sentences

Example:

| “Because I was late, he left a long time ago...”

Understanding this requires remembering earlier tokens far back in the sentence.

Transformer Advantage

- No recurrence
- Uses **attention instead of memory**
- Processes all tokens **in parallel**

- Handles long-range dependencies better
-

Core Idea: Attention

Transformers rely entirely on **attention mechanisms**.

Attention (High-level idea)

- Each word looks at **all other words**
 - Assigns importance scores
 - Focuses on relevant parts of the sentence
-

Self-Attention

Self-attention means:

- Tokens attend to **other tokens in the same sentence**

This allows the model to understand:

- Context
- Relationships
- Dependencies

Example:

| "My name is Rahul and how are you"

The word "*you*" can attend to "*Rahul*" to infer conversational context.

Transformer Architecture

A transformer has **two main components**:

1. Encoder

- Takes input sentence
- Converts tokens into contextual representations

- Learns meaning of each word based on surrounding words

2. Decoder

- Generates output sequence
 - Uses encoder output + self-attention
 - Predicts next token step-by-step
-

Encoder–Decoder Flow

Input:

I like science

Encoder:

- Converts input tokens into context-aware vectors

Decoder:

- Uses encoder output to generate translated / summarized / predicted text
-

Key Components Inside Transformer

Embeddings

- Convert words into vectors
- Same idea as word embeddings, but contextualized via attention

Self-Attention Layer

- Core computation unit
- Computes relationships between all token pairs

Feed Forward Network

- Applied independently to each token

- Adds non-linearity

Comparison with Older Models

Model	Key Property
RNN	Sequential, slow
LSTM	Better memory, still sequential
GRU	Simplified LSTM
Transformer	Parallel, attention-based

Transformers can be seen as:

| "LSTM + Attention → fully attention-based model"

Generative Adversarial Networks (GANs)

What is a GAN?

GANs consist of **two neural networks**:

1. Generator (G)
2. Discriminator (D)

They compete in a **game-like setup**.

Generator (G)

- Takes random noise as input
 - Tries to generate fake data
 - Goal: fool the discriminator
-

Discriminator (D)

- Takes real + fake data

- Outputs probability of being real
 - Goal: detect fake data
-

Adversarial Training (Game Theory View)

- Generator improves by creating more realistic samples
- Discriminator improves by detecting fakes better
- Over time:
 - Generator becomes very good
 - Discriminator cannot tell real vs fake

This is often described as a **minimax game**.

Output Interpretation

Example (multi-class classification):

Classes:

- Dog
- Cat
- Horse
- Lion

Output probabilities:

- Dog → 0.3
- Cat → 0.1
- Horse → 0.2
- Lion → 0.4

Predicted class = **Lion**

Applications of GANs

- Deepfake video generation
 - Image super-resolution
 - Image-to-image translation
 - Drug discovery
 - Material design
 - Data augmentation
-